08-metadata/11/spec/spec_helper.rb
```
RSpec.configure do |config|
  config.filter_run_excluding :jruby_only unless RUBY_PLATFORM == 'java'
end
```

Here, the filter_run_excluding call indicates which examples we're leaving out.

Including Examples

The flip side to that method is filter_run_including, or just filter_run for short. As you might have guessed, this method directs RSpec to run *just* the examples with matching metadata.

This style of filtering is pretty brute-force. If no examples match the filter, RSpec will run nothing at all.

A more generally useful approach is to use filter_run_when_matching. With this method, if nothing matches the filter, RSpec just ignores it. For example, if you haven't marked any specs as *focused* (via the fdescribe/fcontext/fit methods or via the focus: true metadata), the following filter will have no effect:

08-metadata/12/spec/spec_helper.rb
```
RSpec.configure do |config|
  config.filter_run_when_matching :focus
end
```

In this section, we've used RSpec.configure to define example filters. These are permanent settings, baked into your setup code. They'll be in effect every time you run RSpec.

Sometimes, though, you want to filter examples temporarily for just one or two RSpec runs. Editing your spec_helper.rb file every time would get old quickly. Instead, you can use RSpec's command-line interface.

The Command Line

To run just the specs matching a particular piece of metadata, pass the --tag option to rspec. For example, you may want to run just the examples tagged with :fast:

```
$ rspec --tag fast
```

If you prefix the tag name with a tilde (~), RSpec treats the name as an *exclusion* filter. For example, to run all examples that *lack* the :fast tag, you could use the following command:

```
$ rspec --tag ~fast
```